

Project Proposal

Project Title:

Pocket Packet Sniffer

Group Number:

14

Team Profile:

Name	SRN
VIKAS KULAKARNI	PES2UG24CS922
SHIVANAND	PES2UG24CS473
SHIVAKUMARA GP	PES2UG24CS470
SANJITH R	PES2UG24CS445

• Proposed Project Description

Pocket Packet Sniffer is a portable network traffic monitoring tool. It captures packets over a chosen link type — WiFi, Ethernet, or (as a stretch goal) BLE — classifies them, stores captures for later inspection, and presents live summary statistics through a command-line interface, with a web dashboard as an extension. It is designed to run on low-cost hardware such as a Raspberry Pi.

This is a new project with no existing customer; the team is the primary stakeholder, building it to demonstrate sound software design as much as networking functionality. The typical user is a technically literate individual — a student, hobbyist, or small-network operator — who wants visibility into traffic on a network they own or administer, without needing a full tool like Wireshark.

Because this is a Software Engineering course project, the team's explicit priority is a clean, layered design (capture, analysis, storage, presentation, each swappable independently) over maximizing feature count. At the end of the project, the user will be able to:

- Start a packet capture session on a selected network interface
- View live traffic statistics while a capture is running (packet rate, protocol breakdown, top talkers)
- Filter and query previously stored captures by protocol, address, or time window
- Export a session summary report
- (Stretch) Receive an alert when a suspicious traffic pattern is detected
- (Stretch) Run more than one capture backend at once (e.g. Ethernet and BLE together)

• Plan of Work and Product Ownership

Short-Term Plan (Next Few Weeks)

- Weeks 1–2: Finalize requirements and architecture (this proposal + SRS); set up repository, coding standards, and CI skeleton
- Weeks 3–4: Implement the Capture Layer and Analysis Layer for a single backend (Ethernet or WiFi) end-to-end
- Week 5: Implement the Storage Layer and a CLI dashboard consuming it
- Week 6: Integration testing, bug fixing, and documentation of the layer interfaces
- Week 7 onward (if time permits): Stretch goals — second capture backend, Zeek integration, alerting, web dashboard

Team Responsibilities and Feature Ownership

Team Member	Functional Feature(s) Owned	Quality Attribute Focus
Vikas Kulakarni	Packet Capture (Capture Layer, single backend)	Reliability — graceful handling of interface/permission failures
Shivanand	Packet Parsing & Classification (Analysis Layer)	Extensibility — pipeline design so new analyzers plug in cleanly
Shivakumara g p	Capture Storage & Query (Storage Layer)	Maintainability — repository interface decoupled from storage choice
Sanjith R	Live Monitoring Dashboard (Presentation Layer)	Usability — readable output without tool-specific training

- Cross-cutting concerns not tied to a single functional feature — overall architecture review, integration testing, and final report writing — are shared responsibilities across the team, coordinated during weekly check-ins.